

GLAM code

Anatoly Klypin^{1*} and Francisco Prada²

¹ *Astronomy Department, New Mexico State University, Las Cruces, NM, USA*

² *Instituto de Astrofísica de Andalucía (CSIC), Glorieta de la Astronomía, E-18080 Granada, Spain*

28 November 2018

ABSTRACT

Key words: cosmology: Large scale structure - dark matter - galaxies: halos - methods: numerical

Here we present motivation and short description of the GLAM code.

1 THE GLAM CODE

In general, a cosmological PM code consists of three steps to evolve the particles: (1) Using the particle positions $\mathbf{r}$ to obtain the density $\rho_{i,j,k}$ at the nodes of an homogenous 3D-mesh that covers the computational domain, (2) Solve the Poisson equation on the mesh, and (3) advance the particles to a new moment of time.

2 EQUATIONS.

The following parameters are defined at $z = 0$:

$$\Omega_m = \text{density of matter} \quad (1)$$

$$\Omega_\Lambda = \text{contribution of cosmological constant}, \quad (2)$$

$$H_0 = \text{the Hubble constant} \quad (3)$$

$$\Omega_\Lambda + \Omega_m = 1 \quad (4)$$

If we introduce peculiar velocity $\mathbf{v}_{\text{pec}} = a\dot{\mathbf{x}}$ and peculiar momentum $\mathbf{p} = a^2\dot{\mathbf{x}}$, then the equations of motion can be written as:

$$\frac{d\mathbf{p}}{da} = -\frac{\nabla\phi}{a}, \quad \frac{d\mathbf{x}}{da} = \frac{\mathbf{p}}{a^2}. \quad (5)$$

The Friedmann equation defines the rate of expansion:

$$\frac{\dot{a}^2}{a^2} = H_0^2 \left[\frac{\Omega_m}{a^3} + \Omega_\Lambda \right]. \quad (6)$$

The Poisson equation takes form:

$$\nabla^2\phi = 4\pi G a^2 [\rho(\mathbf{x}, t) - \langle \rho \rangle] = 4\pi G a^2 \bar{\rho}_m(t) \delta_m(\mathbf{x}, t), \quad (7)$$

where $\langle \dots \rangle$ implies averaging over space. The matter density contrast $\delta_m(\mathbf{x}, t)$ is:

$$\delta_m(\mathbf{x}, t) = \frac{\rho_m(\mathbf{x}, t) - \bar{\rho}_m(t)}{\bar{\rho}_m(t)}, \quad (8)$$

and $\bar{\rho}_m(t)$ is the average matter density at moment t :

$$\bar{\rho}_m(t) = \frac{\Omega_m \rho_{\text{cr},0}}{a^3}. \quad (9)$$

By design $\langle \delta_m(\mathbf{x}, t) \rangle = 0$ and $\langle \rho_m(\mathbf{x}, t) \rangle = \bar{\rho}_m(t)$.

Now we can re-write the Poisson equation as:

$$\nabla^2\phi = \frac{3\Omega_m}{2a} H_0^2 \delta_m(\mathbf{x}, t), \quad (10)$$

3 DIMENSIONLESS VARIABLES.

Let the box size of simulations be L and the number of grid points in one dimension be N_g . We can introduce dimensionless coordinates $\tilde{\mathbf{x}}$, momenta $\tilde{\mathbf{p}}$ and potentials $\tilde{\phi}$ using the following relations:

$$\mathbf{p} = \tilde{\mathbf{p}} \left(\frac{H_0 L}{N_g} \right), \quad \mathbf{x} = \tilde{\mathbf{x}} \left(\frac{L}{N_g} \right), \quad \phi = \tilde{\phi} \left(\frac{H_0 L}{N_g} \right)^2. \quad (11)$$

Having the dimensionless momenta, we can find the peculiar velocity:

$$\mathbf{v}_{\text{pec}} = 100 \left(\frac{L}{N_g} \right) \left(\frac{\tilde{\mathbf{p}}}{a} \right) \text{ km sec}^{-1}, \quad (12)$$

where we assumed that box size L is given in units of $h^{-1}\text{Mpc}$.

Using these notations, we write the equations of motion and gravity as:

* E-mail: aklypin@nmsu.edu

$$\frac{d\tilde{\mathbf{p}}}{da} = -\left(\frac{H_0}{\dot{a}}\right) \nabla_{\tilde{\mathbf{x}}} \tilde{\phi} \quad (13)$$

$$\frac{d\tilde{\mathbf{x}}}{da} = \left(\frac{H_0}{\dot{a}}\right) \frac{\tilde{\mathbf{p}}}{a^2}, \quad (14)$$

$$\nabla_{\tilde{\mathbf{x}}}^2 \tilde{\phi} = \left(\frac{3\Omega_m}{2a}\right) \delta_m(\tilde{\mathbf{x}}, t), \quad (15)$$

Note that factor $H_0/\dot{a}$ depends only on the expansion parameter:

$$\frac{H_0}{\dot{a}} = \left[\frac{\Omega_m + \Omega_\Lambda a^3}{a} \right]^{-1/2} \quad (16)$$

4 GRAVITATIONAL POTENTIAL

Having the density field $\rho_{i,j,k}$, we can estimate the gravitational potential by solving the Poisson equation, which for clarity we simply write as

$$\nabla^2 \phi = 4\pi G \rho(\mathbf{x}). \quad (17)$$

We start with applying a 3D Fast Fourier Transformation (FFT) to the density field. That gives us the Fourier components on the same grid $\tilde{\rho}_{\mathbf{k}}$, where $\mathbf{k}$ is a vector with integer components in the range $0, 1, \dots, N_g - 1$. Now we multiply the harmonics $\tilde{\rho}_{i,j,k}$ by the Green functions $G(\mathbf{k})$ to obtain the Fourier harmonic amplitudes of the gravitational potential ϕ , i.e.

$$\tilde{\phi}_{i,j,k} = 4\pi G \tilde{\rho}_{i,j,k} G(\mathbf{k}), \quad (18)$$

and then do the inverse FFT to find out the gravitational potential $\phi_{i,j,k}$. Note that all these operations can be organized in such a way that only one 3D-mesh is used – no additional RAM memory is required.

The simplest, but not the best, method to derive the Green functions is to consider $\phi_{i,j,k}$ and $\rho_{i,j,k}$ as amplitudes of the Fourier decomposition of the gravitational potential in the computational volume, and then to differentiate the Fourier harmonics analytically. This gives

$$G_0(\mathbf{k}) = -\frac{1}{\mathbf{k}_x^2 + \mathbf{k}_y^2 + \mathbf{k}_z^2} = -\left(\frac{L}{2\pi}\right)^2 \frac{1}{\mathbf{i}^2 + \mathbf{j}^2 + \mathbf{k}^2}. \quad (19)$$

A better way of solving the Poisson equation that we use in our GLAM code is to start with the finite-difference approximation of the Laplacian ∇^2 . Here we use a the second order Taylor expansion for the spacial derivatives:

$$\begin{aligned} \nabla^2 \phi &= \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} + \frac{\partial^2 \phi}{\partial z^2} \\ &\approx [\phi_{i+1,j,k} - 2\phi_{i,j,k} + \phi_{i-1,j,k} \\ &+ \phi_{i,j+1,k} - 2\phi_{i,j,k} + \phi_{i,j-1,k} \\ &+ \phi_{i,j,k+1} - 2\phi_{i,j,k} + \phi_{i,j,k-1}]/\Delta x^2. \end{aligned} \quad (20)$$

This approximation leads to a large system of linear algebraic equations: $A\phi = 4\pi G\rho$, where ρ is the vector on the right hand side, ϕ is the solution, and A is the matrix of the coefficients. All of its diagonal components are equal to -6, and all 6 nearest off-diagonal components are 1. The solution of this matrix equation can be found by applying the

```
GREENf(1)      = 2.
GREENf(Ngrid) = -2.
DO i=1,Ngrid/2-1
  XX = 2.*COS(P16*i/Ngrid)
  GREENf(2*i)  = XX
  GREENf(2*i+1) = XX
End DO
```

Figure 1. Green functions setup in POTENTfft5 subroutine in PMP2main.f90 file. Parameter $P16 = 2\pi$.

Fourier Transformation. This provides another approximation for the Green functions:

$$G_1(\mathbf{k}) = \frac{\Delta x^2}{2} \left[\cos\left(\frac{2\pi\mathbf{i}}{N_g}\right) + \cos\left(\frac{2\pi\mathbf{j}}{N_g}\right) + \cos\left(\frac{2\pi\mathbf{k}}{N_g}\right) - 3 \right]^{-1}. \quad (21)$$

For small (i, j, k) , eq.(21) gives the same results as eq.(18). However, when (i, j, k) is close to N_g , the finite-difference scheme G_1 provides less suppression for high-frequency harmonics and thus gives a stronger and more accurate force at distances closer to the grid spacing Δx . Hockney & Eastwood (1980) argue that this happens because the finite-difference approximation partially compensates the dumping of short waves that are related with the density assignment.

4.1 Practical implementation.

Subroutine **POTENTfft5** in file **PMP2main.f90** or in **PMP2mainMPI.f90** calculates the dimensionless gravitational potential ϕ on a regular 3D mesh of size $N_g \times N_g \times N_g$. The subroutine's input is a 3D array **FI** of density contrast. The output is an array with the grav.potential that replaces the density contrast. Thus, the original density array is lost and replaced with the potential.

A fragment of the code that defines the Green functions is shown in Figure 1. Parameter **trfi** is the scaling factor in the r.h.s of the poisson equation (15).

5 TIME-STEPPING

Because we start the simulations at a relatively high redshift $z_i \approx 100$, and because the number of time-steps $N_s \approx 100-200$ is not large, the time-stepping scheme should be carefully selected and tuned. In the sense of the accuracy of the time-stepping algorithms, there are two regimes: (1) when fluctuations grow nearly linearly at large redshifts and when the expansion factor a may change substantially over a single time-step, and (2) later moments when fluctuations are in the non-linear regime with a changing very little over a time-step. Both regimes present a challenge for accurate simulations with a small number of time-steps. There are different possibilities in handling these challenges.

At the linear stage of evolution the main concern and the main test is the linear growth of fluctuations. We select a time-stepping scheme which uses a constant time-step at low redshifts $z < z_{\text{limit}}$, but periodically increases the time-step at large redshifts $z > z_{\text{limit}}$. The parameter z_{limit} defines the transition from early quasi-linear to late non-linear regimes. With a resolution of $\Delta x = (0.3-0.5)h^{-1}\text{Mpc}$ in our simulations, some halos may start to collapse and virialise

at $z < z_{\text{limit}}$. This is the stage when we switch the time-stepping to the normal leap-frog scheme with a constant time-step. For our simulations we select $z_{\text{limit}} = 3$.

We can write the time-stepping scheme using a sequence of kick K and drift D operators, which are defined as advances of particle momenta $\mathbf{p}$ and particle coordinates $\mathbf{x}$ from moment a to moment $a + \Delta a$:

$$K(\Delta a, a, \tilde{a}) : \mathbf{p}(a + \Delta a) = \mathbf{p}(a) + \mathbf{g}(\tilde{a})\Delta a, \quad (22)$$

$$D(\Delta a, a, \tilde{a}) : \mathbf{x}(a + \Delta a) = \mathbf{x}(a) + \mathbf{u}(\tilde{a})\Delta a, \quad (23)$$

where $\tilde{a}$ is the moment at which either $\mathbf{u}$ or $\mathbf{g}$ are estimated.

If we start with particle momenta at the time $a_{-1/2} = a_0 - \Delta a/2$ (a half time-step behind the coordinates defined at a_0), and use the notation $a_m = a_0 + m\Delta a$, the standard leap-frog scheme can be written as the following sequence of kick and drift operators:

$$K(\Delta a, a_{-1/2}, a_0)D(\Delta a, a_0, a_{1/2}) \quad (24)$$

$$K(\Delta a, a_{1/2}, a_1)D(\Delta a, a_1, a_{3/2}) \quad (25)$$

$$K(\Delta a, a_{3/2}, a_2)D(\Delta a, a_2, a_{5/2}) \dots \quad (26)$$

When at some moment a_0 we need to increase the time-step by factor 3/2, we do it by making a stronger kick and then by modifying the time-step to the new value of $\Delta a' = 3\Delta a/2$:

$$K(5\Delta a/4, a_{-1/2}, a_0)D(3\Delta a/2, a_0, a_{3/4}) \dots \quad (27)$$

After applying the first pair of kick-drift operands, the normal setup of the leap-frog scheme is restored with the particles momenta behind the coordinates by a half of the new time-step. The code continues the integration of the trajectories with a constant time-step until the moment when $\Delta a/a$ becomes smaller than the minimum value. The time-step is increased again by the factor 3/2, and the process continues.

The truncation error for the variable step scheme can be found similarly to the way how it is done for the standard leap-frog scheme by eliminating the velocities from the scheme, and then by expanding the coordinates in the Taylor series around moment a_0 . This gives $x_{3/2} - (5/2)x_0 + (3/2)x_{-1} = (15/8)g_0\Delta a^2$, and the truncation error ϵ at the moment of time-step increase a_0 is:

$$\epsilon = \frac{5}{16}\dot{g}_0\Delta a^3, \quad (28)$$

which should be compared with the truncation of the constant step leap-frog scheme:

$$\epsilon = \frac{1}{12}\ddot{g}_0\Delta a^4. \quad (29)$$

The truncation error at the moment of modifying the time-step is clearly larger than for the constant-step leapfrog, but it is still a third-order approximation. The reason for that is the selection of the numerical factor 5/4 in the kick operator (eq. 27), which kills the second-order error. These errors are only for a single time-step. The cumulative error for a large number of steps depends on how single-steps errors accumulate. This typically results in scaling the force resolution $\epsilon \propto \Delta a^2$ for the constant time-step. Because there are only very few number of times when the time-step is increased in our code (typically 5-10 times for the total ~ 150 of steps),

the final error is mostly dominated by the cumulative error of the constant-step kicks and drifts.

(i) *Early stages of evolution* $z > z_{\text{limit}}$. It is important to estimate how the terms $\mathbf{u}$ and $\mathbf{g}$, in the right-hand-sides of equations (22-23), evolve with the expansion parameter a at the linear regime. Because there are terms with large powers of a , one may be concerned with the accuracy of the integration of quickly evolving terms. However, when one considers all the terms, the situation is much less alarming. Indeed, in the linear regime the peculiar gravitational potential ϕ does not change with a , and along the particle trajectory $\mathbf{g}(a) \propto a^{1/2}$, leading to $\mathbf{p} \propto a^{3/2}$ and $\mathbf{u} \propto a^0$ (constant). This means that there are no quickly evolving terms in the equations of motions. This slow evolution of the $\mathbf{u}$ and $\mathbf{g}$ terms allows one to periodically increase the time-step without substantial loss of accuracy. We do it by testing the magnitude of $\Delta a/a$. If this ratio falls below a specified value $(\Delta a/a)_{\text{min}}$ (typically $3 - 5 \times 10^{-2}$), the time-step Δa is increased by factor 3/2.

(2) *Late stages of evolution* $z < z_{\text{limit}}$. As fluctuations evolve and become very nonlinear, halos start to form, merge and grow. At this stage the main concern is how accurately the code traces the evolution of dark matter in halos. The number of time-steps is an important factor defining the accuracy. However, the number of steps is just one of the factors: one cannot really find out the required number of steps without specifying the force resolution and without knowing the science application and requirements of the simulations.

Our goal is to generate GLAM simulations that reproduce the dark matter density and velocity fields with the resolution of up to $\sim 1/3 - 1/2 h^{-1}\text{Mpc}$. Peculiar velocities are an integral part of the process implying that redshift distortions should be simulated, not added posteriorly using some analytical prescription. The force resolution and the magnitude of the peculiar velocities set stringent constraints on the allowed time-step.

The largest peculiar velocities $\sim 1000 - 3000 \text{ km sec}^{-1}$ occur in clusters and large galaxy groups. The time-step should be small enough so that per time-step a particle should move by less than a fraction of a cell. Thus, for both stability and accuracy of the integration,

$$\beta \equiv \frac{v\Delta t}{\Delta R} \lesssim 1, \quad (30)$$

where v is the particle velocity, Δt and ΔR are the time-step and the (proper) cell size. Assuming that the time-step is small, we can write $\Delta t = \Delta a/aH(a)$. If $\Delta x = \Delta R/a$ is the comoving cell size, then we can write β in the following form:

$$\beta = \frac{v}{a\Delta x} \frac{\Delta a}{aH(a)} = \frac{v}{\Delta x H_0} \frac{\Delta a}{a} \sqrt{\frac{a}{\Omega_m + \Omega_\Lambda a^3}}. \quad (31)$$

Scaling velocities and resolution to some characteristic values we finally write the condition for selecting the time-step as follows

$$\beta = 10 \left[\frac{\Delta a}{a} \right] \left[\frac{v_{1000}}{\Delta x_{\text{Mpc}}} \right] \sqrt{\frac{a}{\Omega_m + \Omega_\Lambda a^3}} < 1, \quad (32)$$

where v_{1000} is the peculiar velocity in units of 1000 km sec^{-1} and Δx_{Mpc} is the comoving cell size in units of $h^{-1}\text{Mpc}$.

In our simulations the time-step at later moments becomes relatively small with a typical value of $\Delta a/a \approx (0.75 - 1) \times 10^{-2}$, which is sufficient even for fast particles in very massive clusters.

```

FactV = 1.                ! normal step
If(ASTEPA/StepFactor/1.25*AEXPNA.and.AEXPNA<0.3333)Then
  ASTEPA=1.5*ASTEPA      ! increase step
  FactV = 1.2
EndIf
CALL MOVE(FactV)         ! Move particles

```

Figure 2. Changing time-step in program PMP in PMP2main.f90 file. Parameter `StepFactor` defines whether the time step $\Delta a \equiv \text{ASTEPA}$ should be changed or not. The timestep `ASTEPA` is increased by factor 1.5 every time when $\Delta a/a < (4/5)\text{StepFactor}$. This process stops for $a \equiv \text{AEXPNA} > (1/3)$.

```

PCONST = - SQRT(AEXPNA/(0m+0mL*AEXPNA**3))*ASTEPA*0.5/FactV
Ahalf = AEXPNA+ASTEPA/2.
XCONST = ASTEPA/SQRT(Ahalf*(0m+0mL*Ahalf**3))/Ahalf

```

Figure 3. Scaling factors for calculation of changes in momenta (`PCONST`) and coordinates (`XCONST`) in subroutine `Move` in files PMP2main.f90. These are the same factors that appear in r.h.s. of equations eq.(13) and eq.(14). The timestep Δa is `ASTEPA`. Expansion parameter $a \equiv \text{AEXPNA}$. Extra 0.5 factor in `PCONST` is to compensate to the fact that the code in `Move` subroutine produces differences in potential two cells apart, not the derivatives.

5.1 Practical implementation.

Changing coordinates and momenta is done by subroutine `Move(FactV)` in file PMP2main.f90 or PMP2mainMPI.f90. Parameter `FactV` is equal to unity when time-step is constant. It is `FactV=1.2` for the moment when the time-step is increased. Figure 2 show fragment of the code that defines whether the step should be changed or not and what to do if the step is changing. Scaling factors needed to produce changes in coordinates and momenta are shown in Figure 3. These are the same factors that appear in r.h.s. of equations eq.(13) and eq.(14)

6 PARALLELIZATION

Parallelization of GLAM is done with OpenMP directives. Because OpenMP can be applied only to memory on a single computational node, this limits the number of particles and force resolution. This also makes the code faster because the code does not use slow communications across and inside computational nodes required for MPI parallelization. Using only OpenMP directives also makes the code simple and easy to modify. The later is very important because data analysis routines are still being modified and improved.

For solving the Poisson equation the GLAM uses FFT fortran-90 routines for the real-to-real transformations provided by publicly available code `FFT5PACK`. This makes the code portable: no libraries should be installed.

Each OpenMP thread handles N_g^2 1-D FFT transformations. After sweeping the 3-D mesh in two directions the matrix is transposed to improve the data locality. Then the FFT is applied again twice: to complete the 3-D sweep and to start the inverse FFT. The matrix is transposed back, and

the other two FFT sweeps are completed. OpenMP `ATOMIC` directives are applied for density assignments, which slows down the code, but allows it to run in parallel. The motion of particles is a naively parallel part of the code. Overall, the code uses only one 3-D matrix and requires three 3-D FFT passes.

7 PERFORMANCE OF THE GLAM CODE

The GLAM code was tested on a variety of processors (both Intel and AMD) and computer platforms. Results of code timing are given in Table 1 for different hardware configurations and parameters of the simulations. As might have been expected, the Intel processors are about twice faster than AMD when timing results are scaled to CPU-hours per core. However, this is somewhat deceiving because the AMD processors provide about twice more cores. If this is taken into account, then the difference between AMD and Intel processors becomes smaller. For example, a single computational node with four AMD-6376 processors has the same performance as a node with two Intel E5-2680v4 processors when rescaled to the same computational task.

Column 12 in Table 1 provides the CPU time scaled per individual particle. In that respect it is a measure of the efficiency of the parallelization and performance of our PM code. It shows that within $\sim 20\%$ the code scales well for different number particles and mesh sizes.

The computational cost of a GLAM simulation depends on the number of time-steps N_s , the size of the 3D-mesh N_g^3 , and the adopted number of particles N_p^3 . The CPU required to solve the Poisson equation is mostly determined by the cost of performing a single 1D-FFT. Thus, it is proportional to $(N_g \log N_g)^3$. There are some additional costs (e.g., two 3D-matrix transpositions to align the mesh with the memory of individual computational processors), but those are relatively small.

8 MEMORY AND DISK STORAGE REQUIREMENTS

The main memory allocation is for storing 6 numbers per each particle (3 coordinates and three momenta) and for a 3D mesh that stores either density or gravitational potential. If N_p is the number of particles in one dimension and N_g is the number grid points in 1d, then the total memory needed to run the GLAM code is:

$$\text{Memory in Gb} = 22.3 \left(\frac{N_p}{1000} \right)^3 + 3.72 \left(\frac{N_g}{1000} \right)^3. \quad (33)$$

Note that here we assumed a single precision 4-bytes for each quantity. We recommend to use 2-3 times more grid cells in 1d as compared with the number of particles:

$$N_g = (2 - 3)N_p. \quad (34)$$

The amount of disk space to save one snapshot is:

$$\text{Disk space in Gb per snapshot} = 22.3 \left(\frac{N_p}{1000} \right)^3. \quad (35)$$

Table 1. Timing of the GLAM code for different computational systems. The columns give: (1) number of particles N_p , (2) number of grid cells N_g , (3) processor type and number of cores, (4) the total wall-clock time per step in minutes, (5) wall-clock time for the Poisson solver in minutes, (6) advancing particles timing in minutes and (7) density assignment timing in minutes. (8) CPU time per step in minutes, (9) CPU time per step per particle in 10^{-6} seconds, (10) CPU time in hours for a run with 150 time-steps.

(1) N_p	(2) N_g	(3) Processor cores	(4) Total min	(5) Poisson min	(6) Particles min	(7) Density min	(8) CPU step	(9) CPU particle	(10) CPU run
1250 ³	2500 ³	Intel 4114 2.2GHz 2x10	2.10	1.70	0.09	0.24	5.25	1.3	105
2000 ³	5000 ³	Intel E7-8890v4 2.2GHz 4x24	5.00	4.20	0.25	0.53	480	3.6	1200
1200 ³	2400 ³	Intel E5-2680v4 2.4GHz 2x14	1.20	1.02	0.07	0.10	33.6	1.17	84
1200 ³	2400 ³	Intel E5-2680v3 2.5GHz 2x12	1.44	1.22	0.08	0.14	34.5	1.20	86
500 ³	1000 ³	Intel E5-2680v4 2.4GHz 2x14	0.075	0.062	0.0039	0.0088	2.1	1.01	5.2
1000 ³	2000 ³	Intel E5-2680v4 2.4GHz 2x14	0.65	0.55	0.037	0.057	18.2	1.09	45.5
1300 ³	2600 ³	AMD 6174 2.4GHz 4x12	2.23	1.72	0.21	0.30	107	2.92	267
1600 ³	3200 ³	AMD 6376 2.3GHz 4x16	2.44	2.11	0.11	0.22	156	2.28	390

9 STRUCTURE OF DATA FILES

Data are written as unformatted big-endian fortran files.

File Names:

Each snapshot consists of a small header file and a number of large data files. The header file has information on a given snapshot and on the run. The header gives the step number, the expansion parameter and so on. The data files provide information on the coordinates and velocities of particles. The time-step of the snapshot is coded into the file names by using 4 digits of the time-step number. For example, the time-step number 17 gives string 0017, which is incorporated into the file names. The header filename starts with 'PMcrd.' followed by the four digits of the time-step followed by ending '.DAT'.

The same convention is used for the data files. Their names start with 'PMcrs' followed by the file number (0, 1, 2 ...). After that the 4 digits of the step-number and the '.DAT' ending. For example, file names for the time-step 17 may look as: PMcrd.0017.DAT, PMcrs0.0017.DAT, PMcrs1.0017.DAT. The actual number of data files PMcrs depends on the total number of particles.

Format of the header file PMcrd.DAT

The header file has only one record that consists of the following:

- 45-character text string used for identification of a particular simulation. This string will be used in all snapshots of the same simulation. It is just for information.
- 4 numbers each of 4-bytes:
 - 1) expansion parameter,
 - 2) not used,
 - 3) not used
 - 4) step in the expansion parameter
- one integer 4-bytes number: the time-step number.
- 7 real 4-bytes numbers - none used

- 4 integer 4-bytes numbers:

- 1) N_p (number of particles in 1d),
- 2) N_g (grid size in 1d)
- 3) integer 4-bytes number: realization number
- 4) integer 4-bytes number: random seed for the realization

- 3 real 4-bytes numbers:

- 1) Ω_m
- 2) Ω_Λ
- 3) the Hubble constant h .

- 1 integer 8-bytes number: number of particles ($= N_p^3$)
- 100 real 4-byte numbers. Currently only the last is used for the box size in Mpc units

Format of data files:

Each data file PMcrsXXXX.DAT is a direct access fortran unformatted file. Direct access files consist of equal-size records. Each record consists of 6 consecutive arrays of real 4-byte numbers with each array having 1024^2 elements. Arrays are components of coordinates and velocities: x, y, z, vx, vy, vz

10 SNAPSHOTS AND CHECK-POINTS

Special set of files is used for (re)starting simulations. Structure of these files is exactly the same as those of a regular snapshot, but the (re)start files have slightly different names. The header file is always named PMcrd.DAT and the data files are PMcrs0.DAT, PMcrs1.DAT, ... The initial conditions code generates these files. When GLAM code runs, it reads those files and, if required, writes checkpoints to these files. If for some reason the code was terminated, it can be re-started from the last check-pointed set of files. Because the structure of the snapshots is the same as the check-pointed files, a given snapshot can be used to re-start the simulation.

```

#Om_b h^2      = 0.022161
#Om_c h^2      = 0.118890
#Om_Lambda     = 0.692885
#Om_m (1-Om_K-Om_L) = 0.307115
#sigma8 (all matter) = 0.8225142
#at 2
0.10000E+03    0.44975E+03
0.10202E+03    0.45848E+03
0.10408E+03    0.46737E+03
0.10618E+03    0.47644E+03

```

Figure 4. Example of format of file `PkTable.dat` that provides cosmological parameters in the first 5 lines followed by values of the power spectrum. The normalization of the power spectrum in `PkTable.dat` is irrelevant and is ignored by the code. The power spectrum is re-scaled according to required value of σ_8 provided in the input file `Init.dat`.

Simply copy or re-name the snapshot files into check-pointed files.

Information in file `Setup.dat` defines how frequently check-points are made and when snapshots should be saved (if at all).

11 HOW TO COMPILE GLAM CODE

Intel fortran compiler `ifort` should be used to compile the code. Makefile is provided. To compile the code:

- make `PMP2init` – compile the first step of initial conditions code
- make `PMP2start` – compile the first step of initial conditions code
- make `PMP2main` – compile the main code to run a single realization
- make `PMP2mainMPI` – compile code to run many realizations using MPI libraries

12 RUNNING THE GLAM CODE

- Making *one realization at a time* with `PMP2main.exe`.
 - Create a subdirectory. Place there a file `PkTable.dat` with the power spectrum. Create subdirectories that will host different realizations. For example, for two realizations create directories `Run01` and `Run02`.
 - If halo finder will be used, create subdirectories `CATALOGS` in each of `Run01` and `Run02`.
 - Create files `Init.dat` and `Setup.dat` using `PMP2init.exe`. See section on creating initial conditions.
 - Run `PMP2start.exe` from subdirectory `Run01` to create initial conditions. The code will request to input realization number. Give an integer from 0 to 9999.
 - Run `PMP2main.exe` from subdirectory `Run01` to make the simulation and to produce results. Input to the code is a number of time-steps to run. Provide an integer number. If it is too large, the code will stop once $a = 1$ is reached.
 - Proceed to the next realization.
- MMM

13 SETTING INITIAL CONDITIONS

In a directory where data of simulations will be placed, one needs to have file `PkTable.dat` with cosmological parameters

and a table of the power spectrum. The format of the file is the same as output from on-line CAMB code to generate power spectrum for given cosmological model. The structure of the file:

First five lines with cosmological parameters. Each parameter is defined after sign '='. Text before the sign is ignored and serves as description of that parameter. Figure 4 presents an example of format of file `PkTable.dat` that provides cosmological parameters in the first 5 lines followed by values of the power spectrum. The table lists the wavenumbers k in the first column in units of $h\text{Mpc}^{-1}$ and the amplitudes of the power spectrum $P(k)$ in the second column. Values of k must be spaced in constant logarithmic step. Reading routine finds the logarithmic spacing of k values.

The normalization of the power spectrum in `PkTable.dat` is irrelevant and is ignored by the code. The power spectrum is re-scaled according to required value of σ_8 provided in the input file `Init.dat` and then passed to code `PMP2start.exe` as one of parameters in file `Setup.dat`

Setting initial conditions requires two steps. First, one needs to generate file `Init.dat` that lists desired parameters such as the box size, σ_8 , initial redshift, and so on. One can copy the file from another directory, if it was previously generated. If the file `Init.dat` does not exist in the current work directory, it will be generated with default parameters the first time code `PMP2init.exe` is called. Modify the file according to what is desired. Execute `PMP2init.exe` one more time to produce file `Setup.dat`.

Parameters in file `Setup.dat` is what is actually used to set initial conditions. File `Init.dat` was created for convenience, and it is ignored once `Setup.dat` was created. If one needs to modify some parameters of the simulations, they can be changed directly through file `Setup.dat`.

Not all parameters in file `Setup.dat` are created equal. Some parameters are just for information about particular set of simulations. Those are:

- **Cell size** (in $h^{-1}\text{Mpc}$ units). It is defined as the ratio of box size and size of mesh).
- **Particle Mass** (in units of $h^{-1}M_\odot$) is defined by box size, number of particles and Ω_m .
- σ_8 is not used by `PMP2start.exe`. Parameter that defines the amplitude is `DRho/rho in box`.

14 LIST OF GLOBAL VARIABLES AND ARRAYS

Table 2 provides list of global variable available through module `Tools`:

15 DATA ANALYSIS

There are different types of analyses, that can be on-the-flight. The number and the list of redshifts, when the analysis is done, is provided in file `Setup.dat`. Particular type of analysis can be requested by providing value 1 in appropriate line of file `Setup.dat`.

Table 2.

Name	Physical quantity	Description
AEXPN	a	expansion parameter
ASTEP	Δa	current time-step
NROW	N_p	number of particles in 1d
Nparticles	N_p^3	total number of particles
NGRID	N_g	number of mesh points in 1d
Box	L	computational box, $h^{-1}\text{Mpc}$
hubble	h	the Hubble constant units of 100 km/s/Mpc.
Omb	Ω_b	density of baryons
OmL	Ω_Λ	cosmological constant
sigma8	σ_8	amplitude of the power spectrum
Nrealization	–	realization number
StepFactor	$(\Delta a/a)_{\text{init}}$	factor for increasing time-step
XPAR, YPAR, ZPAR	x, y, z	dimensionless coordinates. Each of Nparticles length
VX, VY, VZ	p_x, p_y, p_z	dimensionless momenta Each of Nparticles length
FI	$\delta(\mathbf{x})$	mesh of density contrast or potential (N_g, N_g, N_g) dimension